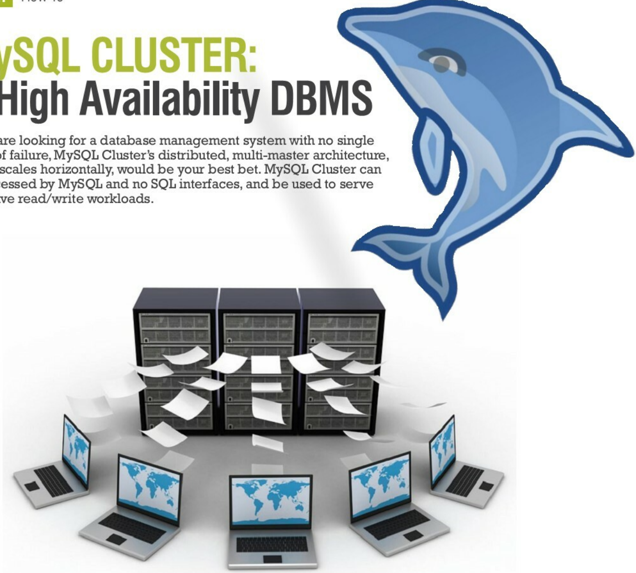
MySQL CLUSTER: A High Availability DBMS

If you are looking for a database management system with no single point of failure, MySQL Cluster's distributed, multi-master architecture, which scales horizontally, would be your best bet. MySQL Cluster can be accessed by MySQL and no SQL interfaces, and be used to serve intensive read/write workloads.



Having set up a DHCP server, I now plan to deploy a DNS server in my network. It is always a better option to have DNS and DHCP on the same server, so that IP addresses allocated by the DHCP server to a particular host can be updated in the DNS database, instantly. If, for some reason, this DNS-DHCP server goes down, it impacts the whole production environment adversely. As a preventive measure, we have to introduce a secondary DNS-DHCP server, which has to be configured in a high availability mode (HA), so that if the primary server goes down, the secondary server takes over and caters to the incoming requests.

PowerDNS is our prime choice for configuring the authoritative DNS server, with MySQL database as the backend, as this combination has its own merits. This set-up handles incoming queries, looks through DNS records in the MySQL database and provides appropriate responses. The DNS servers being HA, the databases in both the servers must always be kept in sync. Moreover, both the DHCP servers work in active-active mode, such that they divide the IP address pool among themselves and cater to the incoming DHCP requests working in tandem. As a result, there are multiple read/writes happening from both the servers in their own MySQL databases. In order to keep both the databases in sync, there has to be such a mechanism, such that if any server makes changes in the database, it should be reflected in the database of the other server and they all maintain the same DNS records.

To create a high availability environment, as mentioned above, MySQL provides two solutions – MySQL replication and the MySQL cluster. Master-slave replication, wherein we have one read/write and one or more read-only slaves, is not useful in this scenario, as the replication is one way (from master to slave). While MySQL master-master replication is one of the alternatives, it is not a good choice, especially when there are multiple masters receiving write requests simultaneously. The big disadvantage of circular replication (A to B, B to C, C to D, and D to A) is that if any node fails, replication halts for subsequent nodes in the chain.